Knowledge Distillation and Deployment of a Command-Parsing Language Model on a Local Cloud

Name: Aman Sen **Roll no.:** 230101026 **Course:** CS3081 Cloud Computing **Repository:** https://github.com/aman-sen/cloud-mini-project (tag `final`)

Convention. *Measured* = produced by a command whose output is stored in `results/` (file named in brackets). *Assumed/expected* = reasoning that I did not measure; marked "(assumption)". Hardware: Apple M1, 8 cores, 8 GB RAM, macOS 26.5 (Docker Desktop VM limited to about 4 GB), single-node k3d cluster.

Task 1. Object storage and parallel teacher labelling

Architecture. `k3d` cluster from `cluster/k3d.yaml` (1 server node; host ports 8080 → Traefik, 8333 → SeaweedFS S3). SeaweedFS (single Pod, PVC) runs in namespace `data` with buckets `datasets` and `models`. The teacher, `phi3.5` served by Ollama (temperature 0, seed 42), runs on the laptop; workers reach it at `host.docker.internal:11434` (`host.k3d.internal` did not resolve on Docker Desktop for Mac). An Indexed Kubernetes Job in namespace `pipeline` starts two Pods in parallel; `SHARD_INDEX` is the Job completion index and `SHARD_COUNT=2`. Credentials come from Kubernetes Secrets generated at deploy time (not in Git).

Teacher prompt (`prompts/teacher_prompt.txt`): states the four keys, the eight intents, the three slot rules, and gives 8 hand-written examples (one per intent, none from the validation set); the command is placed last so Ollama can reuse the cached static prefix. I also pass Ollama's `format: json` so replies are syntactically valid JSON (a decoding constraint, not an edit of the reply); this means rejection reason (a) is expected to be rare and the teacher's JSON validity in Task 2 is partly by construction.

Sharding (≤ ½ page). Worker *k* of *N* reads `commands_pool.jsonl` from the bucket and keeps line *n* (counted from 0) iff *n* mod *N* = *k*. The N residue classes partition {0..2999}, so each command is processed by exactly one worker: no gaps, no repeats, no coordination between workers. Round-robin by line number balances load when commands differ in difficulty. Each worker writes its own object (`datasets/teacher/shard-k.jsonl`), so there are no write conflicts; it re-uploads every 25 records and resumes from its object if the Pod restarts (labelling takes hours). `clean.py` verifies that the union of shards covers every pool id exactly once before cleaning. Both workers share one Ollama instance, so total throughput is bounded by the laptop CPU; parallelism here demonstrates the pattern rather than speeding up. Measured: shard sizes 1500/1500, labelling wall time about 80 min of active Job time in the final run (an earlier run was interrupted when the laptop slept; it had saved 300 records per shard and the workers resumed from those objects), throughput about 28–35 records/min for both workers together [`results/teacher_workers.log`, `results/pipeline_pods.txt`, `results/teacher_shards/`].

Cleaning counts [`results/clean_output.txt`]. Rules are applied in order (a)→(d) and each record is counted once under the first rule that fires; duplicates (same text, smallest id kept) are counted among the records that passed (a)–(d).

input	(a) invalid JSON	(b) wrong keys	(c) bad intent	(d) bad slot	duplicates	kept
3000	0	1	3	59	50	2887

Check: input = a+b+c+d+duplicates+kept → 0+1+3+59+50+2887 = 3000. `train.sha256` = `568d5c60d57432bdac3397f59b0ee5753e0e3817e5e5210897635ed1c423f717` (re-running `clean.py` on the saved shards in `results/teacher_shards/` reproduces it).

Task 2. Training the student (sequence-level distillation)

Setup. `EleutherAI/pythia-70m` (GPT-NeoX, 70 M parameters, Pile-trained, no instruction tuning, Apache-2.0) is fully fine-tuned on the first 1,500 records of `train.jsonl`, text `Command: <cmd>\nJSON: <teacher JSON><eos>`; the loss is computed only on the JSON part. `training/config.yaml` records seed 42, lr 3e-4, batch 8, 6 epochs, max length 96, and library versions; v2 (`config_v2.yaml`) uses seed 7 and 8 epochs. `make train` reads only the config; `make train SUBSET=50` runs the first 50 records. Training time on CPU: v1 8 min 17 s, v2 12 min 33 s. Per-epoch loss: 0.6811, 0.3922, 0.1613, 0.0892, 0.0517, 0.0212 (v1, 6 epochs); 0.6485, 0.3834, 0.1549, 0.1063,

0.0638, 0.0391, 0.0155, 0.0055 (v2, 8 epochs, `training/loss_v2.csv`) [`training/loss.csv`]. Artefacts: `models/v1/`, `models/v2/` in the `models` bucket; release assets `model-v1.tar.gz`, `model-v2.tar.gz`; `models/v1/model.sha256` = `834cd2a1f3c42acfacfc2fb8deb42d5bf4e08cc322cd14b792ecd94ffd97d736` (v2: `999105034edd4d9dbf83225a82351dc94e52d58321491bb342315166c6e07005`) equals `sha256sum` of the Release asset (checked).

Suitability of Pythia-70M (assumption, to be compared with the measurements). The task is narrow and the output format is rigid, so a small model can memorise the format and learn intent cues. The weak point is slot extraction: copying the shortest span exactly needs a copy mechanism and decent tokenisation of numbers/names, which 70 M parameters pre-trained only for next-token prediction on the Pile may handle poorly. A 2,048-token context and BPE vocabulary are more than enough for commands of 5–15 words.

Results on `validation.jsonl` (100 commands; `tools/score.py`, percent) [`results/*_validation.json`, raw outputs `results/*_predictions.jsonl`]:

metric	teacher phi3.5	Pythia-70M untrained	student v1	student v2
json_validity	100	0	99	100
intent_accuracy	87	0	86	79
date_accuracy	83	0	82	78
time_accuracy	86	0	89	88
place_accuracy	91	0	85	84
full_frame	57	0	54	45

What changed. The untrained Pythia-70M scores 0 on every metric because it was never shown the output format (it does not emit the four-key JSON at all). After distillation on 1,500 teacher-labelled commands, student v1 reaches 99 % JSON validity and 54 % full-frame accuracy, i.e. about 95 % of the teacher's 57 % (54/57); on intent it is within one point of the teacher (86 vs 87) and on time it is slightly higher (89 vs 86). The weakest slots are date (82 %) and place (85 %); place shows the largest loss relative to the teacher (91 → 85). Example errors from `student_v1_predictions.jsonl` are copying mistakes: for "november fourteenth" the student produced the invented token "twelveteenth", and "five in morning" was copied where the reference was "five". Student v2 trained longer (final loss 0.0055 vs 0.0212) but is *worse* on full-frame (45 vs 54) and intent (79 vs 86), even though its JSON validity is perfect; I changed both the seed and the epoch count, so I cannot separate overfitting to teacher labels from seed noise, and with 100 validation commands one command is one point (binomial standard error about 5 points), so the 9-point gap is suggestive but I have run only one seed of each. For this reason v1 is the served model and v2 is used for the rolling update. All figures are on 100 commands and are measured, not extrapolated.

Sequence-level vs logit-based distillation (≤ 1 page). *(a) Sequence-level* (Kim & Rush, 2016): the teacher decodes an output sequence for every input (here greedy at temperature 0), and the student is trained with ordinary cross-entropy on that sequence as if it were the label; the student only ever sees the teacher's *text*. *(b) Logit-based* (e.g. TRL's distillation trainer): at every output position the student is trained to match the teacher's whole next-token probability distribution over the vocabulary, typically with a KL-divergence loss on the logits, which transmits "dark knowledge" about the near-miss alternatives, not only the argmax. *(c) Why not here:* the two models do not share a token space: Phi-3.5-mini uses a ~32 k SentencePiece vocabulary and Pythia a ~50 k GPT-NeoX BPE vocabulary, so position *t* and token *i* of one distribution have no counterpart in the other; moreover the teacher is served through Ollama's text-generation API, which returns text rather than the full per-position vocabulary distribution, and storing full distributions for 1,500 examples would also be large. Mapping between vocabularies would need an alignment method that is outside the scope of this project. *(d) Limitation of what I did:* the student's ceiling is the *teacher*, not the ground truth: wherever phi3.5 mis-copies a slot or picks a wrong intent (and passes the cleaning filters), the student is trained to repeat that error; additionally with only 1,500 examples and one decode per command, the student sees no information about teacher uncertainty. The measured teacher accuracy against the gold validation labels (full-frame 57 %, date 83 %, time 86 %) shows that

the teacher itself is wrong on many commands even when its JSON is valid and passes every cleaning rule, so those wrong labels are in `train.jsonl` and the student is trained to imitate them.

Task 3. The inference service in a container

Design. FastAPI/uvicorn (`service/app.py`), port 8000. A background thread downloads `MODEL_URI` (e.g. `s3://models/v1/`) from `S3_ENDPOINT` and loads the model; `/healthz` is 200 whenever the process is alive; `/readyz` is 503 until loading finishes. `POST /parse` builds the training-time prompt `Command: <text>\nJSON:`, decodes greedily (`MAX_NEW_TOKENS`), validates the text against `schema.json` (exactly four keys, intent in the eight, slots string-or-null) and returns exactly the object with headers `X-Model-Version` (`MODEL_VERSION`) and `X-Served-By` (Pod name via the downward API). Invalid output → HTTP 500 `{"error":"invalid_model_output"}`, never broken JSON with 200. `FAULT_INJECT=1` replaces the output with `not json` before validation. A lock serialises generation (1 CPU per Pod). Credentials come only from the `s3-credentials` Secret (`S3_ACCESS_KEY`, `S3_SECRET_KEY`).

Container. `python:3.11.9-slim`, CPU-only `torch==2.5.1` installed from the PyTorch CPU index (the laptop is Apple Silicon, so the image is arm64; the `+cpu` build exists only for x86-64), all dependencies pinned (`service/requirements.txt`), no weights (they are in object storage, `.dockerignore` excludes `models/`). Built with one command: `docker build -f service/Dockerfile -t cmdparser:1.0 .`. Measured image size: 1.26 GB as reported by Docker (260 MB content size) (limit 2 GB) [`results/image_size.txt`].

Evidence. FAULT_INJECT=1 → HTTP 500 with body `{"error":"invalid_model_output"}` [`results/fault_inject.txt`]. 50 sequential requests with `--cpus 1 --memory 1g`: in run 1, 49/50 returned 200 and one (request 27) returned 500 because the student produced invalid JSON (`"time": "in the "right"`), which the service correctly rejected; OOMKilled=false, **peak memory 796.4 MiB** of 1024 MiB [`results/mem_test.log`]. Run 2 on a different set of 50 commands: 50/50 returned 200, OOMKilled=false, **peak 785.0 MiB** [`results/mem_test_run2.log`]. Memory was sampled with `docker stats` every 0.5 s.

Task 4. Kubernetes deployment, scaling and reliability

Manifests (`k8s/30-app.yaml`, namespace `app`): Deployment (2 replicas; requests 500 m CPU/768 Mi, limits 1 CPU/1 Gi; rolling update `maxSurge 1, maxUnavailable 0`; `preStop sleep 5`), Service `cmdparser` (80→8000), Traefik Ingress `/parse`, HPA (min 1, max 2, 70 % CPU of the request; scale-down stabilisation shortened to 60 s for the demo). Liveness `/healthz`; readiness `/readyz`, so a Pod with an unloaded model is not in the Service endpoints.

Check	Result (measured)	Evidence file
`/parse` through Ingress	HTTP 200 with `X-Model-Version: v1` and `X-Served-By: cmdparser-…`	`results/rolling_before.txt`, `results/parse_example.txt`
Service has no ready endpoints while model loads	`kubectl get endpoints` showed `<none>` at 15:20:25 and the Pod was `0/1` Running until 15:20:32 when the endpoint appeared (about 8 s)	`results/endpoints_while_loading.txt`
HPA scale 1→2 under load, back to 1	idle: 1 Pod, CPU 1 %; under load CPU rose to 199 % of the request and Pods went 1→2 at 15:42:37 (second Pod Ready ten seconds later); after load ended (15:46:07) back to 1 Pod at 15:47:36 (about 90 s, with the 60 s scale-down window)	`results/hpa_events.log`, `hpa_describe.txt`, `hpa_top.log`
Rolling update v1→v2 under load	220 requests, 2 failures, **0.91 %** (above the 0.5 % target); both failures were HTTP 500, none were 502/503/connection errors; versions served v1 181, v2 37	`results/rolling_*`

Check	Result (measured)	Evidence file
After `rollout undo`	`X-Model-Version` = v1	`results/rolling_after_undo.txt`
Pod deleted under load	Pod deleted at 15:38:25, replicas back to 2/2 after about 12 s; 133 requests, 2 failures = 1.5 % (≤ 2 %), both HTTP 500	`results/recovery_*`
Cold start, 3 runs (all Pods deleted)	6, 8, 6 s, **median 6.0 s**; logs show the model download from `s3://models/v1/`	`results/coldstart*.{log,txt,json}`
Load balancing (2 Ready Pods, standard command)	Pod A 47 (50 %), Pod B 47 (50 %) (each ≥ 30 %); 95 requests, 1 failure	`results/lb_headers.json`

Cold start is Pod `Ready` transition time minus creation time (1 s resolution), slowest Pod of each run. A first `lb` run inside `scenarios_all.log` was contaminated: three Pod names appeared (48 / 31 / 6 requests), most likely because the HPA, which reconciles the replica count, removed a Pod that I had scaled up manually while CPU was idle and later added another; I reran `lb` alone and report that run. The same effect appears in my second standard load test (below).

Limitations (stated explicitly). The cluster is a *single node* on one laptop: if the machine or the Docker daemon stops, everything stops, so this does not demonstrate fault tolerance across machines (node failure, zone failure, rescheduling to another node). Both replicas and SeaweedFS share the same CPU cores, so a second replica adds availability for a Pod crash/rollout but little extra capacity, and the HPA's "scale-out" mostly shifts CPU from one Pod to another (`kubectl top` shows both Pods at about 1000 m, their full CPU limit, under load, so on this 8-core laptop shared with Docker, Ollama and the load generator the replicas compete for cores; I did not measure a 1-replica baseline, so I cannot state the speed-up). SeaweedFS is a single Pod on a local PVC (single point of failure). The metrics pipeline lags by ~15–30 s, so HPA reacts slowly.

Task 5. Subnetting and network access control

10.20.0.0/16 allocated largest-first (full working in `network/subnets.md`):

Subnet	Hosts	Network/prefix	Usable range	Broadcast	Usable
app	1,000	10.20.0.0/22	10.20.0.1 – 10.20.3.254	10.20.3.255	1,022
public	250	10.20.4.0/24	10.20.4.1 – 10.20.4.254	10.20.4.255	254
data	100	10.20.5.0/25	10.20.5.1 – 10.20.5.126	10.20.5.127	126
management	20	10.20.5.128/27	10.20.5.129 – 10.20.5.158	10.20.5.159	30

NACL (`network/nacl.json`, public subnet). Inbound: 90 deny `198.51.100.0/24` (all protocols); 100 allow TCP 80; 110 allow TCP 443; 120 allow TCP 22 from 10.20.5.128/27; 130 allow TCP 1024–65535 from 10.20.0.0/22 (replies from the app subnet). Outbound: 100 allow TCP 8000 to 10.20.0.0/22; 110 allow TCP 1024–65535 to 0.0.0.0/0 (replies to clients). Everything else falls through to the implicit deny. Measured: `nacl_sim.py … --check` → 6/6 (100 %) [`results/nacl_check.txt`], plus extra hand-made packets in `tests/test_network.py`.

Why separate reply rules; why order matters (≤ ½ page). A NACL is stateless: it does not remember that an inbound request was allowed, so the response is an independent packet that must itself match an allow rule in the other direction. A client's side of a connection uses an ephemeral port (1024–65535), so replies need an outbound rule for ephemeral ports to clients (110) and an inbound ephemeral rule for the answers coming back from the app subnet (130); without them, requests would arrive but responses would be dropped. Rules are checked from the lowest number and the first match decides. Addresses in 198.51.100.0/24 are also inside "0.0.0.0/0", so if the deny had a number *higher* than the allow rules for 80/443 (e.g. 200 vs 100/110) a packet from 198.51.100.9 to port 443

would match the allow first and the deny would never be evaluated. Numbering the deny 90, lower than every allow, makes it win.

Task 6. Reproducibility, testing and CI/CD

One-command deployment. `make up` (fresh clone): venv, `k3d cluster create`, namespaces, generated Secrets, SeaweedFS, model download from the Release with SHA-256 verification, upload to `models/v1`, `v2`, `docker build`, `k3d image import`, apply manifests, wait for rollout. `make down` runs `k3d cluster delete`. Measured: fresh-clone `make up` took 8 min 38 s and ended with both `cmdparser` replicas Ready and a 200 from the Ingress with `X-Model-Version: v1` [`results/make_up.log`]; `make down` deleted the cluster [`results/make_down.log`].

CI (`.github/workflows/ci.yml`), stages in order: *lint* (ruff) → *unit tests* (29 tests: cleaning rules and counts identity, sharding, service/validation/FAULT_INJECT, subnets, NACL, cost, packaging) and *secret scan* (gitleaks 8.18.4, must report nothing) → *evaluate-model* (downloads `model-v1.tar.gz` from the Release, verifies the checksum, runs the model on `validation.jsonl`, `score.py`) → *quality-gate* (`score.py --min-validity 95`). Green run: https://github.com/aman-sen/cloud-mini-project/actions/runs/42193673102. **Failed run with the untrained Pythia-70M (gate fails): https://github.com/aman-sen/cloud-mini-project/actions/runs/42193689236** (JSON validity 0 % < 95). Secret scan result: gitleaks passed (no leaks found).

`AI_USE.md` lists the AI tools used, for what, and how the output was checked.

Task 7. Performance, cost and review

Standard load test (4 users, 2 min, through the Ingress, 2 replicas) [`results/run1/loadtest_stats.csv`, `results/run1/loadtest_headers.json` (run 1); run 2 in `results/loadtest_*`, `results/loadtest_top.log`]:

p50	p95	throughput	requests / failures	error rate	cold start (median, Task 4)
4500 ms	4900 ms	0.88 req/s	104 / 1	0.96 %	6.0 s

This is run 1 (first standard run). I ran the standard test a second time to capture `kubectl top` (run 1 had no sampler; its files are kept in `results/run1/`): run 2 gave p50 4600 ms, p95 9300 ms, 0.77 req/s, 92 requests with 1 failure (1.09 %). Run 2 is worse because the HPA removed one Pod and added another during the run (per-replica counts 51 / 5 / 35), so part of the load hit a cold Pod; run 1 had a clean 52 / 51 split. I report both rather than choosing the better one.

`kubectl top pods -n app` during the test: (run 2, last sample at 17:01:12) Pod A 997 m / 587 Mi, Pod B 996 m / 517 Mi; highest memory seen in the run 851 Mi; CPU about 1000 m per Pod, i.e. at the limit [`results/loadtest_top.log`].

Cost (`rate_card.json`: 3.50 INR per vCPU-hour, 0.45 INR per GiB-hour; billed on limits; `scripts/cost.py`): hourly cost = 2 replicas × (1 × 3.50 + 1 × 0.45) = **7.90 INR/h** (given). Requests per hour = 0.8826 req/s × 3600 = 3,177. Cost per 1,000 requests = 7.90 / 3,177 × 1000 = **2.49 INR** (run 1; run 2 with 0.77 req/s gives 2.84 INR) [`results/cost.txt`, `results/cost_run2.txt`]. This is an estimate; no real charge is incurred.

SLO. “99 % of `/parse` requests return HTTP 200 within 30 s”, window = one standard load-test run. Run 1: total requests 104 → error budget = 1 % × 104 = 1.04 requests; failures = 1; budget used = 96 %. Run 2: 92 requests → budget 0.92; failures 1; budget used = 109 % (SLO missed). All failures are the student’s invalid JSON, so the 99 % target is at the edge of what this model can deliver, while every latency was far below 30 s (max 10.2 s in run 2).

Review (each evidence cell is a measured value):

Component	Evidence (measured)	Limitation found	Improvement proposed
Data and storage	kept 2,887 of 3,000; teacher labelling about 80 min of Job time	teacher errors survive the filters (teacher full-frame 57 %); single-Pod SeaweedFS	replicated storage; filter against a gold dev set

Component	Evidence (measured)	Limitation found	Improvement proposed
Model quality	full_frame v1 54 % vs teacher 57 % (95 % of teacher); validity 99 %	invalid JSON in about 1 % of requests (all HTTP 500s in load tests); v2 worse than v1	more data, logit/feature distillation, constrained decoding
Latency and throughput	p50 4.5 s / p95 4.9 s / 0.88 req/s	1 CPU per Pod, greedy HF generate, lock-serialised, no batching	ONNX/int8, KV-cache reuse, batching
Resource utilisation	about 1000 m CPU and 476–851 Mi per Pod under load; peak 796 MiB in Task 3	CPU request 500 m vs about 1000 m actual; memory request 768 Mi vs up to 851 Mi	raise CPU request, tune requests/limits; smaller dtype
Reliability and recovery	recovery 12 s, 1.5 % failed; rollout 0.91 % failed (2/220, all HTTP 500, no 502/503)	single node; rollout missed the 0.5 % target	multi-node, PodDisruptionBudget, topology spread
Security and cost	2.49 INR per 1,000 requests; secrets only in K8s Secrets, gitleaks clean	S3 admin key shared by all Pods; plain HTTP	per-service least-privilege keys, TLS, NetworkPolicies

References

Kim & Rush (2016), *Sequence-Level Knowledge Distillation*, EMNLP. Bastianelli et al. (2020), *SLURP*, EMNLP (CC BY 4.0). Biderman et al. (2023), Pythia. Hugging Face TRL distillation trainer docs; k3d, Kubernetes, SeaweedFS, Locust documentation.